ILC Lighting App

Session orientation for Manuel San Ramon · July 2026

TO CLAUDE CODE: You have been given this PDF to orient yourself for Manny's session. Read everything on pages 1–2, then the task specs that follow. Greet Manny by name, confirm you understand the project and current tasks by mentioning one or two specifics, tell him Task 1 (clickable links in the Brief page) is the warmup and what the very first step is. Warm tone, short sentences — he wants to be in motion. End with: "Ready to branch and start, or do you want to talk through anything first?"

THE PROJECT

Internal tool for ILC Studios (lighting consultancy). Produces conceptual and design budgets, manages projects, and runs a CRM pipeline. Live at `https://app.ilcstudios.com`, auto-deploys from `main` via AWS Amplify.

Stack: Next.js 14 App Router, Tailwind CSS, Prisma 5.22 (SQLite locally / PostgreSQL in prod), JWT auth, Amplify SSR.

THE TEAM

Name	Role
Mike Kershner	Founder and lead dev. Pushes directly to main.
Sean McCauley	Manny's reviewer. Merges all Manny's PRs.
Manny (you)	Part-time dev ~10 hr/week. Always works on manny/* branches.

MANNY'S MANDATORY WORKFLOW — EVERY SESSION

Step	Command / Action
1	git pull (always first — never skip)
2	git checkout -b manny/<short-description>
3	Edit code, test with npm run dev:3000, iterate
4	git commit, then git push (pre-push hooks run ~25 sec — never use --no-verify)
5	gh pr create, then ping Sean to review. Never push to main directly.

PRE-PUSH HOOKS (NEVER BYPASS)

Six checks run automatically on every push: dynamic-pages, postgres-migrations, schema-sync, `npm test` (38 scenarios + 112K fuzz), check-secrets, push-size. If one fails, fix the underlying issue — **never use** `--no-verify`.

CRITICAL RULES

Rule	Detail
force-dynamic on every page	Every file under src/app/(app)/ must export: export const dynamic = 'force-dynamic' Without it the page cra
schema.prisma stays sqlite	provider = "sqlite" must never be changed — Amplify patches it to postgresql at build time.
Leave the budget engine alone	src/lib/calculations.js is validated. Do not touch unless a task explicitly says to.
Auth on every API route	Every route handler starts with: const { user, error } = await requireAuth(request)

LOCAL DEV LOGIN

```
npm run dev:3000

Email:    admin@ilcstudios.com
Password: ILCadmin2024!
```

CURRENT STATE (as of 2026-07-07)

Tasks 1–9 from the last two sprints all merged to main. Now on Tasks 1–4 in this document — all CRM polish, pure frontend. No schema changes, no engine changes.

KEY FILES FOR THIS SPRINT

File	Purpose
src/app/(app)/crm/brief/BriefClient.jsx	Task 1 — add clickable project name links
src/app/(app)/crm/page.jsx	Task 2 — add _count to the Prisma select
src/app/(app)/crm/CRMClient.jsx	Tasks 2, 3, 4 — notes badge, prob edit, CSV export
src/app/api/projects/[id]/route.js	Task 3 — PATCH endpoint (already accepts pipelineProbability)

Tasks 1–4 overview

All four tasks are CRM polish. No schema changes, no engine changes. Work in order: 1 → 2 → 3 → 4. Tasks 1 and 2 are quick warmups (~30 min and ~1.5 hrs).

Task	Branch	File(s)	Effort
1 — Brief page clickable links	manny/brief-clickable-links	BriefClient.jsx	~30 min
2 — Notes count badge	manny/crm-notes-count	crm/page.jsx + CRMClient.jsx	~1.5 hr
3 — Probability quick-edit	manny/crm-prob-edit	CRMClient.jsx	~2-3 hr
4 — CSV export	manny/crm-csv-export	CRMClient.jsx	~2 hr

Task 1 — Brief page: clickable project names

Branch: `manny/brief-clickable-links` | File: `src/app/(app)/crm/brief/BriefClient.jsx` | ~30 min

The Morning Brief page shows stale projects and recent wins as tables, but project names are plain text. Mike can't click through. One-line change per table makes the page actionable.

What to change

1. In the **stale projects table** (the `stale.map` block), find the `<td>` that renders `{p.name}` and wrap it:

```
<a
  href={`/project/${p.id}`}
  className="font-semibold text-slate-900 hover:text-blue-600 underline"
>
  {p.name}
</a>
```

2. Apply the same link pattern to **project names in the recent wins section**.

The `id` field is already in the Prisma select in `brief/page.jsx` — no server changes needed.

Acceptance criteria

* Clicking a project name in the stale table navigates to /project/[id].
* Same for the recent wins section.
* No other changes to the page. Pre-push hooks pass.

Task 2 — Notes count badge on ProjectCard

Branch: `manny/crm-notes-count` | Files: `crm/page.jsx` + `CRMClient.jsx` | ~1.5 hours

The last sprint added a '+ Note' button on each kanban card (Task 4). There's no way to tell at a glance if a project already has notes. A small count badge closes the loop.

Step 1 — add _count to crm/page.jsx

Inside the `prisma.project.findMany({{ select: {{ ... }} }})` block, add one line:

```
_count: { select: { noteLinks: true } },
```

`noteLinks` is the relation name on Project (declared as `noteLinks NoteProject[] @relation("ProjectNoteLinks")` in schema.prisma). Each count is the number of notes linked to the project.

Step 2 — show the count in ProjectCard (CRMClient.jsx)

Find the `ProjectCard` function (search for `// ■■ Project Card`). After the `{{!noteOpen ? ... <NoteInput>}}` block at the bottom of the return, add:

```
{(project._count?.noteLinks ?? 0) > 0 && (
  <p className="text-[10px] text-slate-400 mt-0.5 select-none">
    {project._count.noteLinks} note{project._count.noteLinks !== 1 ? 's' : ''}
  </p>
)}
```

Acceptance criteria

* Cards with existing notes show "N notes" below the '+ Note' button.
* Cards with zero notes show nothing extra.
* No API changes, no schema changes. Pre-push hooks pass.

Task 3 — Probability quick-edit on ProjectCard

Branch: `manny/crm-prob-edit` | File: `src/app/(app)/crm/CRMClient.jsx` | ~2–3 hours

The probability bar and percentage on each card are display-only. Editing requires navigating into the project. Click-to-edit on the card saves three clicks.

Architecture — read before writing

* `projects` state and `setProjects` live in `CRMClient` (~line 363).
* `ProjectCard` receives `project` as a prop — it cannot update parent state without a callback.
* `handleDrop` already shows the optimistic-update pattern: call `setProjects` immediately, then fire the PATCH.
* API: `PATCH /api/projects/[id]` already accepts `pipelineProbability` and clamps it to 0–100 server-side.

Step 1 — add handleProbabilityChange to CRMClient (near handleDrop)

```
async function handleProbabilityChange(id, value) {
  const clamped = Math.max(0, Math.min(100, Number(value)));
  setProjects(prev => prev.map(p =>
    p.id === id ? { ...p, pipelineProbability: clamped } : p
  ));
  try {
    await fetch(`/api/projects/${id}`, {
      method: 'PATCH',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ pipelineProbability: clamped }),
    });
  } catch { /* optimistic already applied; fail silently */ }
}
```

Step 2 — thread onProbabilityChange to ProjectCard

Wherever `ProjectCard` is rendered (inside `KanbanColumn` and inside `ListView`), add the prop. Also thread it through `KanbanColumn`'s props.

```
// In CRMClient's JSX:
<KanbanColumn ... onProbabilityChange={handleProbabilityChange} />
<ListView     ... onProbabilityChange={handleProbabilityChange} />

// In KanbanColumn:
function KanbanColumn({ ..., onProbabilityChange }) {
  // pass it down:
  <ProjectCard ... onProbabilityChange={onProbabilityChange} />
}
```

Step 3 — add editing state and inline input inside ProjectCard

```
const [editingProb, setEditingProb] = useState(false);

// Replace the probability <span> with this:
{editingProb ? (
  <input
    type="number" min="0" max="100"
```

```
    defaultValue={project.pipelineProbability ?? 0}
    autoFocus
    className="w-10 text-[10px] border border-slate-200 rounded px-1
               focus:outline-none focus:ring-1 focus:ring-orange-400"
    onBlur={e => {
      setEditingProb(false);
      onProbabilityChange(project.id, e.target.value);
    }}
    onKeyDown={e => {
      if (e.key === 'Enter')  e.target.blur();
      if (e.key === 'Escape') setEditingProb(false);
    }}
    onClick={e => e.stopPropagation()}
  />
) : (
  <span
    className="text-[10px] text-slate-400 cursor-pointer
               hover:text-slate-600 hover:underline"
    title="Click to edit"
    onClick={e => { e.stopPropagation(); setEditingProb(true); }}
  >
    {project.pipelineProbability ?? 0}%
  </span>
)}
```

Critical: `e.stopPropagation()` on both the span click and the input click — without it, clicking to edit opens the project detail panel.

Acceptance criteria

* Clicking the % on a card opens a number input (0–100).
* Blur or Enter saves via PATCH and updates the bar immediately (optimistic).
* Escape cancels without saving.
* Clicking the input does NOT open the project panel.
* Pre-push hooks pass.

Task 4 — CRM CSV export

Branch: `manny/crm-csv-export` | File: `src/app/(app)/crm/CRMClient.jsx` | ~2 hours

Mike occasionally needs the pipeline as a spreadsheet. All data is already in client state — no API needed. A download button in the CRM toolbar serializes the current filtered view.

Step 1 — add handleExportCSV to CRMClient (near handleDrop)

```
function handleExportCSV() {
  const sf     = statusFieldFor(pipeline);
  const stagef = stageFieldFor(pipeline);
  const headers = [
    'Project', 'Client', 'Stage', 'Status',
    'Revenue', 'Probability', 'Expected Order',
  ];
  const rows = filtered.map(p => [
    p.name,
    p.client?.name || '',
    p[stagef] || '',
    p[sf] || '',
    revenueFor(p, pipeline)
      ? `$${revenueFor(p, pipeline).toLocaleString()}` : '',
    `${p.pipelineProbability ?? 0}%`,
    p.expectedOrderDate
      ? new Date(p.expectedOrderDate).toLocaleDateString() : '',
  ]);
  const csv = [headers, ...rows]
    .map(r => r.map(v =>
      `"${String(v).replace(/"/g, '""')}"`
    ).join(','))
    .join('\n');
  const blob = new Blob([csv], { type: 'text/csv' });
  const url  = URL.createObjectURL(blob);
  const a    = document.createElement('a');
  a.href     = url;
  a.download = `crm-${pipeline.toLowerCase()}-`
             + new Date().toISOString().slice(0, 10) + '.csv';
  a.click();
  URL.revokeObjectURL(url);
}
```

Step 2 — add the Export CSV button in the toolbar

Find the CRM toolbar JSX (search for `// ■■ Main CRMClient` and look for the area with the pipeline tab switcher and 'New Project' button). Add a low-key secondary button:

```
<button
  onClick={handleExportCSV}
  className="text-xs border border-slate-200 text-slate-600
             hover:bg-slate-50 px-2.5 py-1.5 rounded-lg"
>
  Export CSV
</button>
```

Acceptance criteria

* Clicking 'Export CSV' downloads a .csv file of the current filtered pipeline view.
* Filename includes pipeline name and today's date (e.g. `crm-procurement-2026-07-07.csv`).
* Columns: Project, Client, Stage, Status, Revenue, Probability, Expected Order.
* File opens cleanly in Excel or Google Sheets.
* No API routes added. Pre-push hooks pass.

Operating reminder (for Claude): Tell Manny what to do — not how to do it technically. He approves proposed changes; Claude handles git, tests, and pushes. Manny never pushes to main — always via a manny/* branch and PR to Sean. If pre-push hooks fail, diagnose and fix. Never --no-verify.